DataBase Assignment : 2

Abdullah Sonija 24k-0013

Database Assignment: 2

Question 01:

A.

```
SELECT DISTINCT c.customer_name
FROM Customers c
JOIN Purchases p ON c.customer_id =
p.customer_id
JOIN Purchase_Details pd ON
p.purchase_id = pd.purchase_id
JOIN Books b ON pd.book_id = b.book_id
WHERE b.price > (SELECT AVG(price)
FROM Books);
```

B.

```
SELECT b1.title
FROM Books b1
WHERE b1.price > (
SELECT AVG(b2.price)
FROM Books b2
WHERE b1.publisher =
```

```
b2.publisher
);
```

C.

```
SELECT DISTINCT customer_name
FROM Customers
WHERE customer_id IN (SELECT
customer_id FROM Purchases)
AND customer_id NOT IN (
SELECT p.customer_id
FROM Purchases p
JOIN Purchase_Details pd ON
p.purchase_id = pd.purchase_id
JOIN Books b ON pd.book_id = b.book_id
WHERE b.publisher = 'Pearson'
);
```

D.

```
SELECT title
FROM Books
WHERE book_id NOT IN (SELECT book_id
FROM Purchase_Details);
```

E.

```
SELECT DISTINCT
```

```
a.author_name
FROM Authors a
JOIN Book_Authors ba ON a.author_id =
ba.author_id
JOIN Books b ON ba.book_id = b.book_id
WHERE b.price > (SELECT AVG(price)
FROM Books);
```

F.

```
SELECT p.customer_id, SUM(pd.quantity)
AS total_quantity
FROM Purchases p
JOIN Purchase_Details pd ON
p.purchase_id = pd.purchase_id
GROUP BY p.customer_id
HAVING SUM(pd.quantity) > 10;
```

Question 02:

A.

```
SELECT p.passenger_name, b.bus_name,
r.source_city, r.destination_city
FROM Tickets t
JOIN Passengers p ON t.passenger_id =
p.passenger_id
JOIN Buses b ON t.bus_id =
```

```
b.bus_id
JOIN Routes r ON t.route_id =
r.route_id;
```

B.

```
SELECT p.passenger_name,
t.travel_date, t.fare
FROM Tickets t
JOIN Passengers p ON t.passenger_id =
p.passenger_id;
```

C.

```
SELECT b.bus_name,
COUNT(t.passenger_id) AS
total_passengers
FROM Buses b
LEFT JOIN Tickets t ON b.bus_id =
t.bus_id
GROUP BY b.bus_id, b.bus_name;
```

D.

```
SELECT r.source_city,
r.destination_city, p.passenger_name
FROM Tickets t
JOIN Routes r ON t.route_id =
```

```
r.route_id
JOIN Passengers p ON t.passenger_id =
p.passenger_id;
```

E.

```
CREATE VIEW PassengerTravelSummary
AS
SELECT p.passenger_name, b.bus_name,
r.source_city, r.destination_city,
t.travel_date
FROM Tickets t
JOIN Passengers p ON t.passenger_id =
p.passenger_id
JOIN Buses b ON t.bus_id = b.bus_id
JOIN Routes r ON t.route_id =
r.route_id;
```

F. DDL Operations.

```
ALTER TABLE Passengers ADD
contact_number VARCHAR(15);
ALTER TABLE Tickets MODIFY fare
DECIMAL(10,2);
ALTER TABLE Buses DROP COLUMN
capacity;
DROP TABLE
```

Routes;

Question 03:

a.

WorkoutProgram: ProgramID (PK), ProgramName, Duration, DifficultyLevel, CoordinatorName

Trainer: TrainerID (PK), TrainerName, ContactNumber

FitnessCenter: CenterID (PK), CenterName

b.

Trainer conducts WorkoutProgram: Many-to-Many (M:N). A trainer can run multiple programs, and a program can be conducted by multiple trainers.

FitnessCenter hosts WorkoutProgram: Many-to-Many (M:N). A center can host multiple programs, and a program can be offered at multiple centers.

c.

Associative Entity: Trainer_Assignment

Attributes: TrainerID (FK), ProgramID

(FK), Role.
Note: The Primary Key is a composite key made of both Foreign Keys (TrainerID, ProgramID).

d.
Why CoordinatorName is not a separate entity
In this simplified design, each Workout Program is managed by exactly one Coordinator, and the system does not require storing any additional details about the coordinator like their phone number or email. Since it is a simple 1:1 attribution with a single string value, creating a separate table would unnecessarily complicate the database and force extra JOIN operations. Representing it as a single attribute (CoordinatorName) inside the WorkoutProgram table is an efficient denormalization for this specific use case.

Product: ProductID (PK), ProductName, Category, UnitPrice, StockQuantity.
Outlet: OutletID (PK), OutletName, City, Province, ManagerName.
Customer: CustomerID (PK), FirstName, LastName, Street, City, Province, PostalCode.
Customer_Phone: PhoneNumber (PK), CustomerID (FK).
Customer_Email: EmailAddress (PK), CustomerID (FK).
Sale: SaleID (PK), SaleDate, QuantitySold, UnitPrice, TotalPrice, PaymentStatus, PaymentMethod.
Outlet_Stocks_Product (Associative Entity for M:N):
Cardinality: M:N. (An outlet stocks multiple products; a product is available in multiple outlets).

Resolution: An associative entity named Inventory or Outlet_Stock with composite PK (OutletID,

ProductID).

Customer makes Sale:
Cardinality: 1:M. (A customer can have multiple sales, but a specific sale belongs to only one customer).

Outlet processes Sale:
Cardinality: 1:M. (An outlet conducts multiple sales, but a specific sale happens at one outlet).

Sale involves Product:
Cardinality: M:1. (A Sale record has a single ProductID, meaning one sale record is one product line item. A product can be linked to many sales).

Question 05:
a.
Insertion Anomaly: We cannot add data about a new Fact-Checking Agency (its Name and Phone) or a new AI Model into the system unless they are immediately tied to a specific news

Update Anomaly: If a Fact-Checking Agency updates its AgencyPhone, we must update every single row in the database where that agency verified an article. Missing even one row results in data inconsistency.

Deletion Anomaly: If we delete the record of an article, and it happens to be the only article a specific Journalist ever submitted, we permanently lose all information about that Journalist from the database.

b.
IDs are unique identifiers.
Verifications are unique events.
A platform hosts an article.

FD1:
ArticleID -> ArticleTitle, SubmissionDate, JournalistID, PlatformID
FD2: JournalistID -> JournalistName,

JournalistEmail
FD3: PlatformID -> PlatformName
FD4: AIModelID -> AIModelName, ModelVersion
FD5: AgencyID -> AgencyName, AgencyPhone
FD6: VerificationID -> ArticleID, AgencyID, VerificationDate, VerificationOutcome, ConfidenceScore

c.
Articles: ArticleID (PK), ArticleTitle, SubmissionDate, JournalistID (FK), PlatformID (FK)
Journalists: JournalistID (PK), JournalistName, JournalistEmail
Platforms: PlatformID (PK), PlatformName
AI_Models: AIModelID (PK), AIModelName, ModelVersion
Agencies: AgencyID (PK), AgencyName, AgencyPhone
Verifications: VerificationID (PK), ArticleID (FK), AgencyID (FK), VerificationDate, VerificationOutcome,

ConfidenceScore
Article_AI_Analysis: ArticleID (FK), AIModelID (FK)

d.
Repeated AgencyPhone: If an agency verifies 1,000 articles, its phone number is stored 1,000 times. This wastes storage space and causes an Update Anomaly if the phone number changes.

Repeated AIModelName and ModelVersion: Since an AI model analyzes multiple articles, its configuration data repeats for every single article it touches. This makes the table massively bloated and susceptible to Insertion/Deletion Anomalies if models are added/removed independently of articles.